

- Description

This module manages a bank loan approval system, ensuring that:

1. Loan applications are evaluated based on income, requested amount, and credit score.
2. Applicants with sufficient income and a high credit score receive approval.
3. Loan requests exceeding the applicant's eligibility are either partially approved or rejected.
4. A log of all loan decisions is maintained for financial tracking and review.

This system ensures fair and data-driven loan approvals while minimizing financial risk.

- Specification

module BankLoanApproval

var income, requestedAmount, approvedAmount: real

var creditScore: integer

var loanDecisionLog: sequence of string

process evaluateLoan(income: real, requestedAmount: real, creditScore: integer)
approvedAmount: real

functional scenario form:

creditScore < 600 && approvedAmount = 0 and loanDecisionLog =
loanDecisionLog $\oplus$ ("Loan Denied: Low Credit Score"), Loan denied due to low credit
score.

||

requestedAmount <= income * 5 && approvedAmount = requestedAmount
and loanDecisionLog = loanDecisionLog $\oplus$ ("Full Loan Approved"), Full loan approved.

||

requestedAmount <= income * 6 && approvedAmount = income * 5 and
loanDecisionLog = loanDecisionLog $\oplus$ ("Partial Loan Approved"), Partial loan approved.

||

requestedAmount > income * 6 && approvedAmount = 0 and loanDecisionLog
= loanDecisionLog $\oplus$ ("Loan Denied: Exceeds Limit"), Loan denied due to exceeding limit.

end process

end module

- Java Code

```
package code;
import java.util.*;
class LoanApplication {
    private String applicantName;
    private double income;
    private double requestedAmount;
    private int creditScore;
    private double approvedAmount;
```

```

    public LoanApplication(String applicantName, double income, double
requestedAmount, int creditScore) {
        this.applicantName = applicantName;
        this.income = income;
        this.requestedAmount = requestedAmount;
        this.creditScore = creditScore;
        this.approvedAmount = evaluateLoan();
    }
    private double evaluateLoan() {
        double maxLoanAmount = income * 5;
        if (creditScore < 600) {
            return 0.0;
        } else if (requestedAmount <= maxLoanAmount) {
            return requestedAmount;
        } else if (requestedAmount <= maxLoanAmount * 1.2) {
            return maxLoanAmount;
        } else {
            return 0.0;
        }
    }
    public boolean isApproved() {
        return approvedAmount > 0;
    }
    public String getApprovalDetails() {
        if (isApproved()) {
            return String.format("Applicant: %s | Requested: $%.2f | Ap
proved: $%.2f | Credit Score: %d",
                applicantName, requestedAmount, approvedAmount, cre
ditScore);
        } else {
            return String.format("Applicant: %s | Requested: $%.2f | Lo
an Denied | Credit Score: %d",
                applicantName, requestedAmount, creditScore);
        }
    }
}

class LoanApprovalSystem {
    private List<String> loanDecisionLog;
    public LoanApprovalSystem() {
        this.loanDecisionLog = new ArrayList<>();
    }
    public void processLoanApplication(String name, double income, doub
le amount, int creditScore) {

```

```

        LoanApplication application = new LoanApplication(name, income,
amount, creditScore);
        System.out.println(application.getApprovalDetails());
        if (application.isApproved()) {
            loanDecisionLog.add(application.getApprovalDetails());
        }
    }
    public void printLoanDecisionLog() {
        System.out.println("\nLoan Decision Log:");
        for (String log : loanDecisionLog) {
            System.out.println(log);
        }
    }
}
public class c20_BankLoanSystem {
    public static void main(String[] args) {
        LoanApprovalSystem system = new LoanApprovalSystem();
        // testcase-VT:
//      system.processLoanApplication("Alice", 50000, 200000, 720);
//      system.processLoanApplication("Bob", 40000, 250000, 680);
//      system.processLoanApplication("Charlie", 30000, 180000, 500);
//      system.processLoanApplication("David", 60000, 700000, 750);
//      system.processLoanApplication("Eve", 40000, 220000, 680);

        // testcase-FT:
//      system.processLoanApplication("", 44953.21598647061, 15132.906
611333936, 33149);
//      system.processLoanApplication("?", 44953.21598647061, 781140.7
19111334, 33149);
//      system.processLoanApplication("?dlice", 58223.6782719807, -
499.9998640002624, 0);
//      system.processLoanApplication("?Idlic", 58222.633361816406, -
499.99999937997376, 26572181);
//      system.processLoanApplication("", -499.9999999999933, -
500, 0);

        // testcase-Z3:
//      system.processLoanApplication("Alice", 50000, 200000, 197);
//      system.processLoanApplication("Bob", 40000, 250000, -1);
//      system.processLoanApplication("Charlie", 30000, 180000, 601);
//      system.processLoanApplication("David", 60000, 700000, 599);
//      system.processLoanApplication("Eve", 40000, 220000, 601);

        // testcase-UVT:

```

```

        system.processLoanApplication("Alice", 50000, 200000, 720);
        system.processLoanApplication("Bob", 40000, 250000, 680);
        system.processLoanApplication("Charlie", 30000, 180000, 500);
        system.processLoanApplication("David", 60000, 700000, 750);
        system.processLoanApplication("Eve", 40000, 220000, 680);

        system.printLoanDecisionLog();
    }
}

```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.7 %	14	828	842
src	1.7 %	14	828	842
code	1.7 %	14	828	842
c20_BankLoanSystem.java	100.0 %	14	0	14
> c20_BankLoanSystem		0	0	0
> LoanApplication	100.0 %	10	0	10
> LoanApprovalSystem	100.0 %	4	0	4

3. Vibration testing instruction coverage: 98.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.2 %	197	16,492	16,689
src	1.2 %	197	16,492	16,689
code	1.2 %	197	16,492	16,689
c20_BankLoanSystem.java	98.5 %	197	3	200
> c20_BankLoanSystem	92.5 %	37	3	40
> LoanApplication	100.0 %	111	0	111
> LoanApprovalSystem	100.0 %	49	0	49

4. Fuzz testing branch coverage: 92.9%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.5 %	13	829	842
src	1.5 %	13	829	842
code	1.5 %	13	829	842
c20_BankLoanSystem.java	92.9 %	13	1	14
> LoanApplication	90.0 %	9	1	10
> c20_BankLoanSystem		0	0	0
> LoanApprovalSystem	100.0 %	4	0	4

5. Fuzz testing instruction coverage: 97.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.2 %	195	16,494	16,689
src	1.2 %	195	16,494	16,689
code	1.2 %	195	16,494	16,689
c20_BankLoanSystem.java	97.5 %	195	5	200
> c20_BankLoanSystem	92.5 %	37	3	40
> LoanApplication	98.2 %	109	2	111
> LoanApprovalSystem	100.0 %	49	0	49

6. Z3 branch coverage: 85.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.4 %	12	828	840
src	1.4 %	12	828	840
code	1.4 %	12	828	840
c20_BankLoanSystem.java	85.7 %	12	2	14
> LoanApplication	80.0 %	8	2	10
> LoanApprovalSystem	100.0 %	4	0	4
> c20_BankLoanSystem		0	0	0

7. Z3 instruction coverage: 96%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.9 %	192	21,903	22,095
▼ src	0.9 %	192	21,903	22,095
▼ code	0.9 %	192	21,903	22,095
▼ c20_BankLoanSystem.java	96.0 %	192	8	200
> c20_BankLoanSystem	92.5 %	37	3	40
> LoanApplication	95.5 %	106	5	111
> LoanApprovalSystem	100.0 %	49	0	49

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.7 %	14	826	840
▼ src	1.7 %	14	826	840
▼ code	1.7 %	14	826	840
▼ c20_BankLoanSystem.java	100.0 %	14	0	14
> LoanApplication	100.0 %	10	0	10
> LoanApprovalSystem	100.0 %	4	0	4
> c20_BankLoanSystem		0	0	0

9. Unimproved V-method instruction coverage: 98.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.9 %	197	21,898	22,095
▼ src	0.9 %	197	21,898	22,095
▼ code	0.9 %	197	21,898	22,095
▼ c20_BankLoanSystem.java	98.5 %	197	3	200
> c20_BankLoanSystem	92.5 %	37	3	40
> LoanApplication	100.0 %	111	0	111
> LoanApprovalSystem	100.0 %	49	0	49